

Contents

Tutorial 1	1	4	Improving Naïve Search	2
1 Subsequence	1	5	Knuth–Morris–Pratt Example	3
2 Longest Common Subsequence	1	6	Longest Palindromic Subsequence	4
3 Naïve Search	2			

For the purpose of these exercises we assume that array indices start at 0, i.e. an array A of n integers has indices $0, \dots, n-1$. A string is considered to be an array of characters.

1. Subsequence Describe an efficient algorithm to check whether a string $\mathbf{a} = (a_1, \dots, a_n)$ contains $\mathbf{b} = (b_1, \dots, b_m)$ as an ordered (but not necessarily continuous) subsequence string. What if we want to count the number of such occurrences?

To detect a substring, we need only read $\mathbf{a}$ while keeping track of characters from $\mathbf{b}$ seen so far.

Algorithm: SUBSEQ(a, b)

```

j ← 0
for i = 0 to n - 1 do
    if ai = bj then
        j ← j + 1
    if j = m then
        return True
return False

```

It is not hard to see that this terminates after at most n constant time loops. Hence $\mathcal{O}(n)$.

Counting non-overlapping occurrences can be done by resetting the j and printing an occurrence rather than simply returning True.

Overlapping occurrences are harder due in no small part to their potentially exponential number.

2. Longest Common Subsequence If a sequence of characters $\mathbf{c} = (c_1, \dots, c_k)$ is a subsequence of both $\mathbf{a} = (a_1, \dots, a_n)$ and $\mathbf{b} = (b_1, \dots, b_m)$, we call $\mathbf{c}$ *common* to $\mathbf{a}$ and $\mathbf{b}$. Note that subsequence need not be continuous.

Describe an efficient algorithm for finding a longest common subsequence of two given sequences of characters.

We describe a dynamic algorithm to find a longest common subsequence. We use a two dimensional table dp of size $(n+1) \times (m+1)$. At the end of the algorithm, $dp[i, j]$ should be equal to the maximum length of subsequences common to $\mathbf{a}[1 : i] = (a_1, \dots, a_i)$ and $\mathbf{b}[1 : j] = (b_1, \dots, b_j)$. We compute the values in dp row by row:

- For $i = 0$ or $j = 0$, we have $dp[i, j] = 0$ because $\mathbf{a}[1 : 0]$ and $\mathbf{b}[1 : 0]$ are empty.
- For $i > 0$ and $j > 0$, we assume that previous values have already been computed. Then, there are three choices to consider for $dp[i, j]$: $dp[i-1, j]$, $dp[i, j-1]$ and, if $a_i = b_j$, $dp[i-1, j-1] + 1$ because any common subsequence can be extended by the character $a_i = b_j$. We pick the one with a highest value.

The following algorithm computes the length of a longest common subsequence using the above approach. Note that we do not actually need to keep to whole table dp in memory all the time, it is enough to keep the current line and the previous line.

Algorithm: LCS DYNAMIC($\mathbf{a}, \mathbf{b}$)

Input: Sequences $\mathbf{a} = (a_1, \dots, a_m)$ and $\mathbf{b} = (b_1, \dots, b_n)$.

Output: The length of a longest common subsequence of $\mathbf{a}$ and $\mathbf{b}$.

```

dp ← new table of size (m + 1) × (n + 1)
for j ← 0 to n do
    dp[0, j] ← 0
for i ← 1 to m do
    dp[i, 0] ← 0
    for j ← 1 to n do
        dp[i, j] ← max(dp[i - 1, j], dp[i, j - 1])
        if ai = bj then
            dp[i, j] ← max(dp[i, j], dp[i - 1, j - 1] + 1)
return dp[m, n]

```

Tutorial 1 (continued)

A memoized recursion is also possible, but the whole `dp` table will need to be stored in memory.

Algorithm: LCSMEMO(**a**, **b**)

Input: Sequences $\mathbf{a} = (a_1, \dots, a_n)$ and $\mathbf{b} = (b_1, \dots, b_m)$.

Output: The length of a longest common subsequence of $\mathbf{a}$ and $\mathbf{b}$.

```
dp ← empty  $(n + 1) \times (m + 1)$  table
for  $i = 0$  to  $m$  do
  dp[ $i, 0$ ] ← 0
for  $j = 1$  to  $n$  do
  dp[ $0, j$ ] ← 0

def LCSUB( $i, j$ ):
  if dp[ $i, j$ ] empty then
    dp[ $i, j$ ] ← max(LCSUB( $i - 1, j$ ), LCSUB( $i, j - 1$ ))
    if  $a_i = b_j$  then
      dp[ $i, j$ ] ← max(dp[ $i, j$ ], LCSUB( $i - 1, j - 1$ ))

return LCSUB( $m, n$ )
```

3. Longest Palindromic Subsequence Denote by α^{-1} the reversed string α so that $\alpha^{-1}[i] = \alpha[-i]$ for each index i . We say that a string is a *palindrome* if $\alpha^{-1} = \alpha$.

Describe a dynamic algorithm that finds a longest palindromic subsequence in a given sequence of characters $a = (a_1, \dots, a_n)$.

This is very similar to the longest common subsequence. We again evaluate a table `dp` of size $n \times n$ which will, upon termination, contain the length of a longest palindromic subsequence of $\mathbf{a}[i : j] = (a_i, \dots, a_j)$ in cell `dp[ $i, j$ ]`.

The result is then in `a[1,  $n$ ]`. Note that we only use the upper triangle of the table (above the main diagonal) and memory may be saved by storing only the previous and current diagonal lines.

Algorithm: LONGESTPALIN(**a**)

Input: Sequence $\mathbf{a} = (a_1, \dots, a_n)$.

Output: The length of a longest palindromic subsequence of $\mathbf{a}$.

```
dp() ← new table of size  $n \times m$ 
for  $i \leftarrow 1$  to  $n$  do
  dp[ $i, i$ ] ← 1 // Single character is always a palindrome
for  $i \leftarrow 1$  to  $n - 1$  do
  if  $a_i = a_{i+1}$  then // Two characters must be same to form a palindrome
    dp[ $i, i + 1$ ] ← 2
  else
    dp[ $i, i + 1$ ] ← 0
for  $k \leftarrow 2$  to  $n$  do
  for  $i \leftarrow 0$  to  $n - k$  do
    dp[ $i, i + k$ ] ← max(dp[ $i + 1, i + k$ ], dp[ $i, i + k - 1$ ])
    if  $a_i = a_{i+k}$  and dp[ $i, i + k$ ] < dp[ $i + 1, i + k - 1$ ] + 2 then
      /* We can extend a palindromic subsequence of  $a_{i+1, i+k-1}$  by character
          $a_i = a_{i+k}$  from both sides. */
      dp[ $i, i + k$ ] ← dp[ $i + 1, i + k - 1$ ] + 2
return dp[1,  $n$ ]
```

4. Naïve Search The naïve string matching algorithm prints starting indices of all occurrences of a pattern string P within text string T and can be formulated as follows.

Tutorial 1 (continued)

Algorithm: Naïve string matching algorithm

Input: Pattern string $P[0 \dots m-1]$, text string $T[0 \dots n-1]$.

Output: Prints starting positions of pattern P within text T .

```
for  $i \leftarrow 0$  to  $n - m$  do
     $k \leftarrow 0$ 
    while  $k < m$  and  $P[k] = T[i + k]$  do
         $k \leftarrow k + 1$ 
    if  $k = m$  then
        print  $i$ 
```

Show that, for every m and n with $m < n$, there is a text T of length n and a pattern P of length m such that the naïve string matching performs $\Omega(nm)$ steps (character to character comparisons). Even if P has no occurrence in T .

$P = \text{aaa} \dots \text{aab}$ and $T = \text{aaa} \dots \text{aaa}$. Each of the $n - m$ iterations of the for loop procs the while loop which evaluates m times— $\Omega(mn - m^2)$.

If $m \ll n$ (or more precisely, $m \in o(n)$), then this is asymptotic to $\Omega(mn)$.

5. Improving Naïve Search Assuming that the pattern string P contains no repeated character, modify the naïve string matching algorithm to work in linear time.

What if no character occurs more than two times?

If each character of the pattern is distinct then, after the while loop terminates, we know that none of the characters it scanned can be the head of an occurrence.

Algorithm: Distinct string matching algorithm

Input: Pattern string $P[0 \dots m-1]$ with distinct characters, text string $T[0 \dots n-1]$.

Output: Prints starting positions of pattern P within text T .

```
 $i \leftarrow 0$ 
while  $i < n - m$  do
     $k \leftarrow 0$ 
    while  $k < m$  and  $P[k] = T[i + k]$  do
         $k \leftarrow k + 1$ 
    if  $k = m$  then
        print  $i$ 
     $i \leftarrow i + k$ 
```

This algorithm evaluates each character of T once— $\mathcal{O}(n)$. If we process P to validate that it has no repeat characters, then $\mathcal{O}(m + n)$.

Instead of skipping to the end of the failed search, we store the index of the first repeated head character j and jump i to j .

Tutorial 1 (continued)

Algorithm: 2-Distinct string matching algorithm

Input: Pattern string $P[0 \dots m-1]$ with 2-bounded repetitions, text string $T[0 \dots n-1]$.

Output: Prints starting positions of pattern P within text T .

```
 $i \leftarrow 0$ 
while  $i < n - m$  do
   $j \leftarrow 0, \quad k \leftarrow 0$ 
  while  $k < m$  and  $T[i+k] = P[k]$  do
     $k \leftarrow k + 1$ 
    if  $T[i+k] = P[0]$  and  $j < i$  then
       $j \leftarrow i + k$ 
  if  $k = m$  then
     $\text{print } i$ 
  if  $j > i$  then
     $i \leftarrow j$ 
  else
     $i \leftarrow i + k$ 
```

Here, each character of T is processed $\#$ -of-repeated- $P[0]$ times. If this number is bounded, then we get linear time in n (with a potentially quite large constant).

This is like naively storing, from the KMP Automaton, the earliest failure edge which points to the head of P . By storing more backwards edges and using them more intelligently, KMP achieves more general linear time complexity.

A refresher of definitions:

- The KMP (Knuth-Morris-Pratt) automaton is at state α after processing a string σ iff α is the longest suffix of σ which is also a prefix of the needle ι .
 - When the KMP automaton is at state α and does not match the next character, it takes a back edge $Z(\alpha)$ which goes to the longest proper suffix of α which is also a prefix of ι .
-

6. Knuth–Morris–Pratt Example Construct the search automata for the words *kokos* and *ananas*.